Tugas Pertemuan 6.1: Integrity Constraint

Mata Kuliah: Sistem Basis Data (2026)

Prodi: Sarjana Terapan Teknik Informatika POLBAN

Kelas: 2B

NIM: 251524048

Nama Lengkap: Ghaisan Khoirul Badruzaman

1. Business Rules

Business rules untuk mengelola data mahasiswa (students):

Kode	Business Rule
BR01	Setiap mahasiswa memiliki student_id berupa NIM 9 digit angka. student_id wajib diisi dan tidak boleh sama dengan mahasiswa lain, karena kolom ini yang membedakan satu mahasiswa dengan mahasiswa lainnya.
BR02	Nama depan (first_name) wajib diisi. Nama belakang (last_name) boleh kosong, karena ada mahasiswa yang namanya hanya satu kata.
BR03	Setiap mahasiswa wajib punya email. Email tidak boleh dipakai oleh dua mahasiswa dan harus berformat nama@domain (ada tanda @ dan titik setelahnya).
BR04	Jenis kelamin (gender) wajib diisi dan hanya boleh bernilai 'M' (male) atau 'F' (female), huruf kapital.
BR05	Tempat lahir (place_of_birth) wajib diisi.
BR06	Tanggal lahir (date_of_birth) wajib diisi dan harus masuk akal, yaitu antara 1 Januari 1950 dan 31 Desember 2012.
BR07	IPK (gpa) boleh kosong untuk mahasiswa baru yang belum punya nilai. Kalau diisi, nilainya 0.00 sampai 4.00 dengan dua angka di belakang koma.

2. Tabel students

Tabel students dibuat di database kampus pada PostgreSQL. Constraint langsung ditulis di dalam CREATE TABLE sesuai business rules di atas.

Judul
Membuat database kampus dan tabel students
Syntax

`sudo -u postgres psql`
 	
`\pset pager off`
 	
`\pset null '[null]'`
 	
`CREATE DATABASE kampus;`

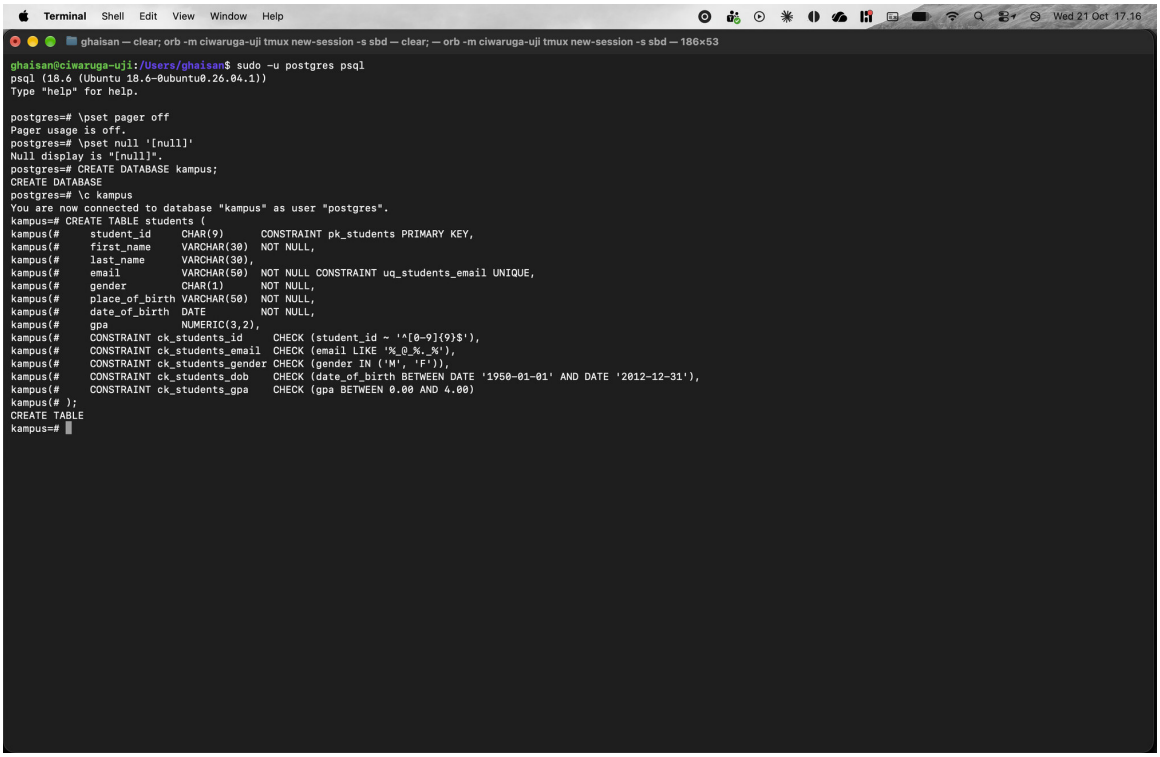
```
\c kampus

CREATE TABLE students (
    student_id      CHAR(9)       CONSTRAINT pk_students PRIMARY KEY,
    first_name      VARCHAR(30)   NOT NULL,
    last_name       VARCHAR(30),
    email           VARCHAR(50)   NOT NULL CONSTRAINT uq_students_email UNIQUE,
    gender          CHAR(1)       NOT NULL,
    place_of_birth VARCHAR(50)   NOT NULL,
    date_of_birth   DATE          NOT NULL,
    gpa             NUMERIC(3,2),
    CONSTRAINT ck_students_id      CHECK (student_id ~ '^[0-9]{9}$'),
    CONSTRAINT ck_students_email   CHECK (email LIKE '%_@_%._%'),
    CONSTRAINT ck_students_gender CHECK (gender IN ('M', 'F')),
    CONSTRAINT ck_students_dob     CHECK (date_of_birth BETWEEN DATE '1950-01-01'
    AND DATE '2012-12-31'),
    CONSTRAINT ck_students_gpa     CHECK (gpa BETWEEN 0.00 AND 4.00)
);
```

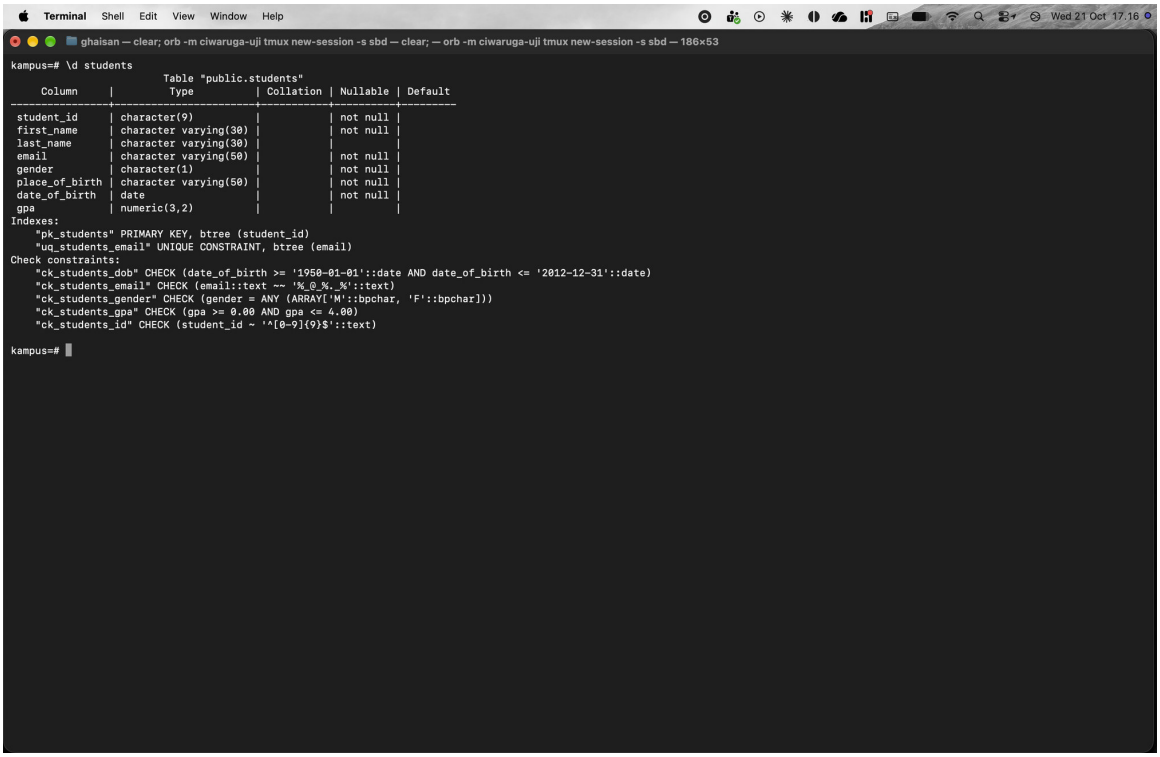
Keterangan

Database kampus dibuat, lalu tabel students dibuat dengan delapan kolom sesuai instruksi. Semua constraint diberi nama (pk_, uq_, ck_) supaya kalau ada pelanggaran, pesan error langsung menyebutkan constraint mana yang menolak.
NOT NULL ditulis langsung di samping kolom, sedangkan CHECK ditulis di bawah sebagai table constraint. CREATE TABLE berhasil tanpa error.

Hasil Eksekusi (Screenshot)



Judul
Struktur tabel students beserta constraint-nya
Syntax
`\d students`
Keterangan
\d students menampilkan struktur tabel. Kolom Nullable menunjukkan kolom yang NOT NULL, bagian Indexes menunjukkan PRIMARY KEY (pk_students) dan UNIQUE (uq_students_email), dan bagian Check constraints menunjukkan kelima CHECK. Jadi semua constraint sudah tersimpan di tabel. PostgreSQL menulis ulang beberapa CHECK ke bentuk lain, misalnya BETWEEN menjadi >= AND <= dan IN menjadi = ANY (ARRAY[...]), tapi artinya tetap sama. student_id juga tercatat not null walaupun tidak ditulis NOT NULL, karena PRIMARY KEY otomatis tidak boleh NULL.
Hasil Eksekusi (Screenshot)



3. Rule Kolom gender

Kolom gender bertipe CHAR(1) NOT NULL dan diberi constraint `ck_students_gender CHECK (gender IN ('M', 'F'))`. Jadi gender wajib diisi dan hanya bisa bernilai 'M' atau 'F'. Pembuktiannya ada di Percobaan 2.

4. Penerapan Integrity Constraints

Pemetaan constraint pada tabel students ke jenis integrity constraint dan business rule:

Jenis	Constraint	Kolom	Aturan	BR
Domain (NULL)	NOT NULL	first_name, email, gender, place_of_birth, date_of_birth	kolom wajib diisi	BR02-BR 06
Domain (NULL)	boleh NULL	last_name, gpa	kolom boleh kosong	BR02, BR07
Domain (CHECK)	ck_students_id	student_id	9 digit angka	BR01
Domain (CHECK)	ck_students_email	email	format nama@domain	BR03
Domain (CHECK)	ck_students_gender	gender	hanya 'M' atau 'F'	BR04
Domain (CHECK)	ck_students_dob	date_of_birth	1950-01-01 sampai 2012-12-31	BR06
Domain (CHECK)	ck_students_gpa	gpa	0.00 sampai 4.00	BR07
Key	pk_students (PRIMARY KEY)	student_id	unik, tidak boleh ada NIM ganda	BR01
Key	uq_students_email (UNIQUE)	email	unik, satu email satu mahasiswa	BR03
Entity Integrity	pk_students (PRIMARY KEY)	student_id	primary key tidak boleh NULL	BR01

5. Percobaan DML INSERT dan UPDATE

Setiap constraint diuji dengan INSERT atau UPDATE yang melanggar aturan, lalu dicek apakah PostgreSQL menolaknya. Percobaan dianggap berhasil kalau data yang salah ditolak dan data yang benar tetap bisa masuk.

Judul

Percobaan 1: INSERT data yang memenuhi semua constraint

Syntax

```
INSERT INTO students
VALUES ('251524901', 'Andi', 'Pratama', 'andi.pratama@contoh.ac.id', 'M',
    'Bandung', DATE '2006-03-14', 3.75),
       ('251524902', 'Siti', 'Rahmawati', 'siti.rahma@contoh.ac.id', 'F',
    'Garut', DATE '2005-11-02', 3.90);

-- last_name dan gpa boleh NULL
INSERT INTO students (student_id, first_name, email, gender, place_of_birth,
    date_of_birth)
VALUES ('251524903', 'Rizky', 'rizky@contoh.ac.id', 'M', 'Cimahi', DATE
    '2006-07-21');

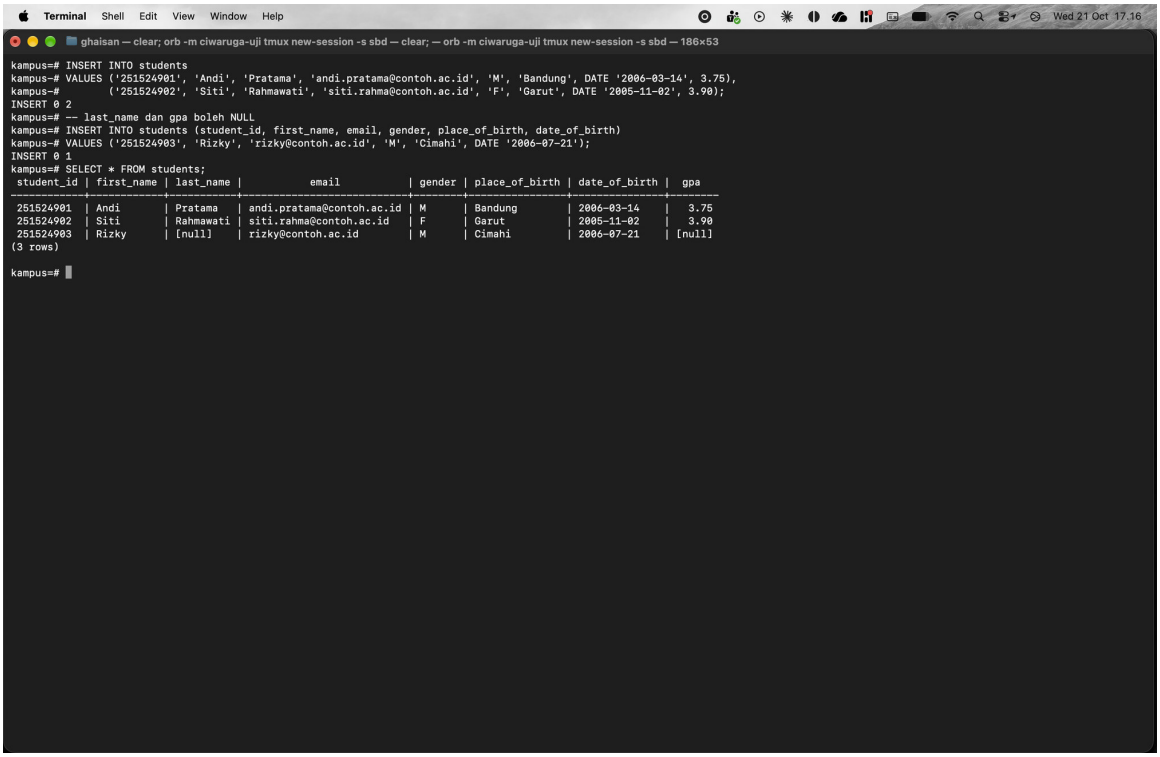
SELECT * FROM students;
```

Keterangan

Dua mahasiswa dimasukkan sekaligus dengan satu INSERT, lalu satu mahasiswa lagi tanpa last_name dan gpa. Ketiganya berhasil masuk (INSERT 0 2 dan INSERT 0 1) karena semua nilainya memenuhi constraint. Kolom yang tidak diisi otomatis bernilai NULL dan ditampilkan sebagai [null].

Hasil percobaan: berhasil. Data yang sesuai business rule bisa masuk, dan last_name serta gpa memang boleh NULL (BR02, BR07). Ketiga data ini dipakai untuk percobaan berikutnya.

Hasil Eksekusi (Screenshot)



Judul

Percobaan 2: rule gender, INSERT dan UPDATE dengan nilai selain 'M' dan 'F'

Syntax

```
-- gender 'L'
INSERT INTO students
VALUES ('251524904', 'Budi', 'Santoso', 'budi.s@contoh.ac.id', 'L', 'Bogor', DATE
    '2006-01-09', 3.10);

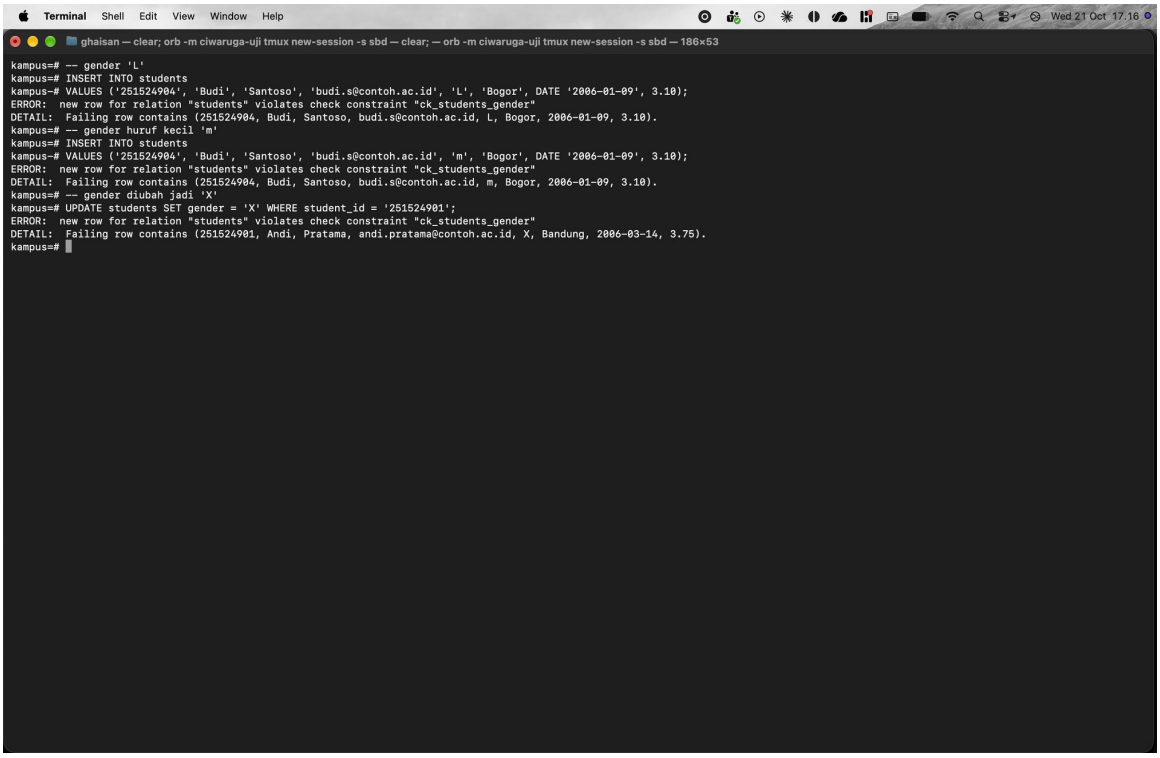
-- gender huruf kecil 'm'
INSERT INTO students
VALUES ('251524904', 'Budi', 'Santoso', 'budi.s@contoh.ac.id', 'm', 'Bogor', DATE
    '2006-01-09', 3.10);

-- gender diubah jadi 'X'
UPDATE students SET gender = 'X' WHERE student_id = '251524901';
```

Keterangan

Gender 'L' ditolak oleh ck_students_gender karena yang diizinkan hanya 'M' dan 'F'.
Gender 'm' (huruf kecil) juga ditolak, karena perbandingan teks di PostgreSQL membedakan huruf besar dan kecil, jadi 'm' tidak sama dengan 'M'.
UPDATE gender menjadi 'X' ditolak oleh constraint yang sama, jadi rule ini juga tidak bisa dilewati lewat UPDATE.
Hasil percobaan: berhasil. Rule gender hanya 'M' dan 'F' (BR04) terbukti berjalan pada INSERT maupun UPDATE.

Hasil Eksekusi (Screenshot)



Judul

Percobaan 3: domain constraint NOT NULL

Syntax

```
-- first_name NULL
INSERT INTO students
VALUES ('251524905', NULL, 'Lestari', 'lestari@contoh.ac.id', 'F', 'Cianjur',
    DATE '2006-05-30', 3.40);

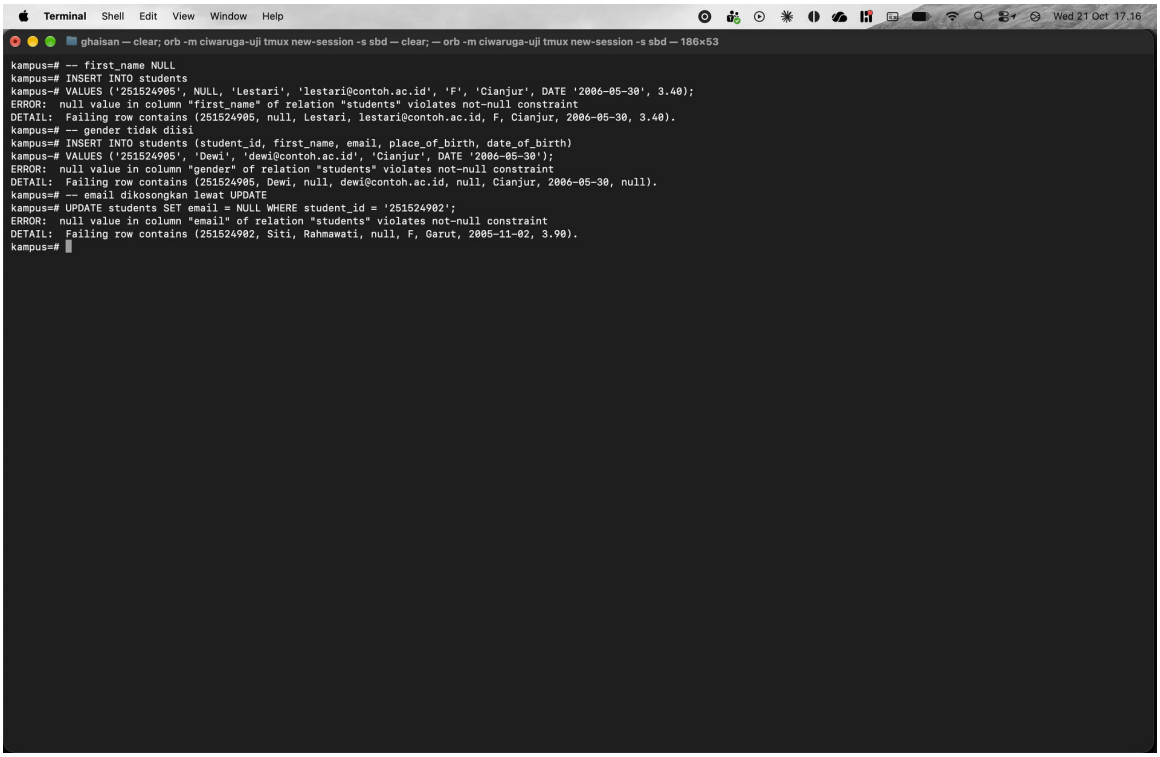
-- gender tidak diisi
INSERT INTO students (student_id, first_name, email, place_of_birth,
    date_of_birth)
VALUES ('251524905', 'Dewi', 'dewi@contoh.ac.id', 'Cianjur', DATE '2006-05-30');

-- email dikosongkan lewat UPDATE
UPDATE students SET email = NULL WHERE student_id = '251524902';
```

Keterangan

INSERT dengan first_name NULL ditolak dengan pesan violates not-null constraint pada kolom first_name.
INSERT tanpa menyebut kolom gender juga ditolak. Kolom yang tidak disebut otomatis diisi NULL karena tidak punya DEFAULT, padahal gender NOT NULL.
UPDATE email menjadi NULL ditolak, jadi data yang sudah ada juga tidak bisa dikosongkan.
Hasil percobaan: berhasil. Domain constraint NOT NULL pada kolom wajib (BR02 sampai BR06) terbukti berjalan.

Hasil Eksekusi (Screenshot)



Judul

Percobaan 4: domain constraint CHECK (format NIM, email, tanggal lahir, dan IPK)

Syntax

```
-- student_id bukan 9 digit angka
INSERT INTO students
VALUES ('25152A906', 'Dewi', 'Lestari', 'dewi@contoh.ac.id', 'F', 'Cianjur', DATE
    '2006-05-30', 3.40);

-- email tanpa @
INSERT INTO students
VALUES ('251524906', 'Dewi', 'Lestari', 'dewi.contoh.ac.id', 'F', 'Cianjur', DATE
    '2006-05-30', 3.40);

-- tanggal lahir di luar rentang
INSERT INTO students
VALUES ('251524906', 'Dewi', 'Lestari', 'dewi@contoh.ac.id', 'F', 'Cianjur', DATE
    '1945-08-17', 3.40);

-- gpa lebih dari 4.00
INSERT INTO students
VALUES ('251524906', 'Dewi', 'Lestari', 'dewi@contoh.ac.id', 'F', 'Cianjur', DATE
    '2006-05-30', 4.25);

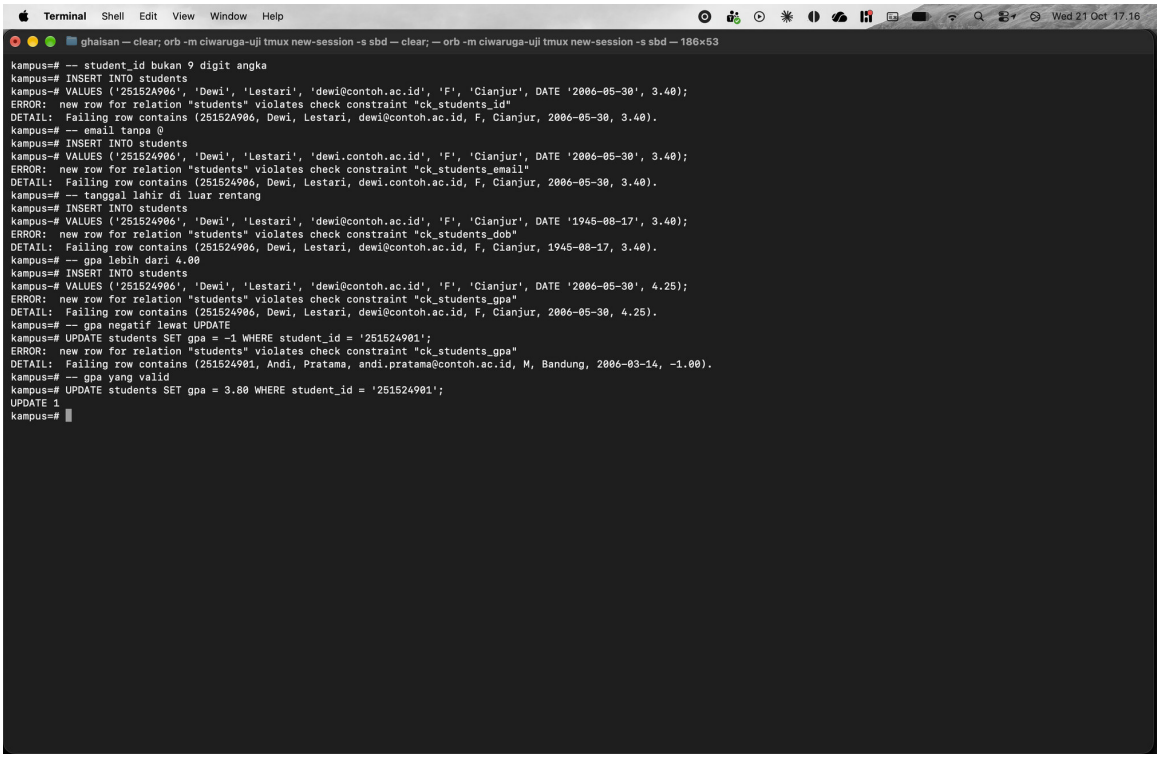
-- gpa negatif lewat UPDATE
UPDATE students SET gpa = -1 WHERE student_id = '251524901';

-- gpa yang valid
UPDATE students SET gpa = 3.80 WHERE student_id = '251524901';
```

Keterangan

student_id '25152A906' ditolak oleh ck_students_id karena ada huruf A. Pola '^[0-9]{9}$' artinya dari awal sampai akhir hanya boleh berisi 9 angka.
Email 'dewi.contoh.ac.id' ditolak oleh ck_students_email karena tidak ada tanda @.
Tanggal lahir 1945-08-17 ditolak oleh ck_students_dob karena di luar rentang 1950-01-01 sampai 2012-12-31.
gpa 4.25 ditolak oleh ck_students_gpa karena lebih dari 4.00. Tipe NUMERIC(3,2) sebenarnya masih bisa menampung 4.25 (maksimal 9.99), jadi yang menolak memang CHECK, bukan tipe datanya. UPDATE gpa menjadi -1 juga ditolak, sedangkan UPDATE gpa menjadi 3.80 berhasil (UPDATE 1) karena masih di rentang 0.00 sampai 4.00.
Hasil percobaan: berhasil. Semua domain constraint CHECK menolak data yang salah dan tetap menerima data yang benar.

Hasil Eksekusi (Screenshot)



Judul

Percobaan 5: key constraint PRIMARY KEY dan UNIQUE

Syntax

```
-- student_id ganda
INSERT INTO students
VALUES ('251524901', 'Dewi', 'Lestari', 'dewi@contoh.ac.id', 'F', 'Cianjur', DATE
    '2006-05-30', 3.40);

-- email ganda
INSERT INTO students
VALUES ('251524906', 'Dewi', 'Lestari', 'siti.rahma@contoh.ac.id', 'F',
    'Cianjur', DATE '2006-05-30', 3.40);

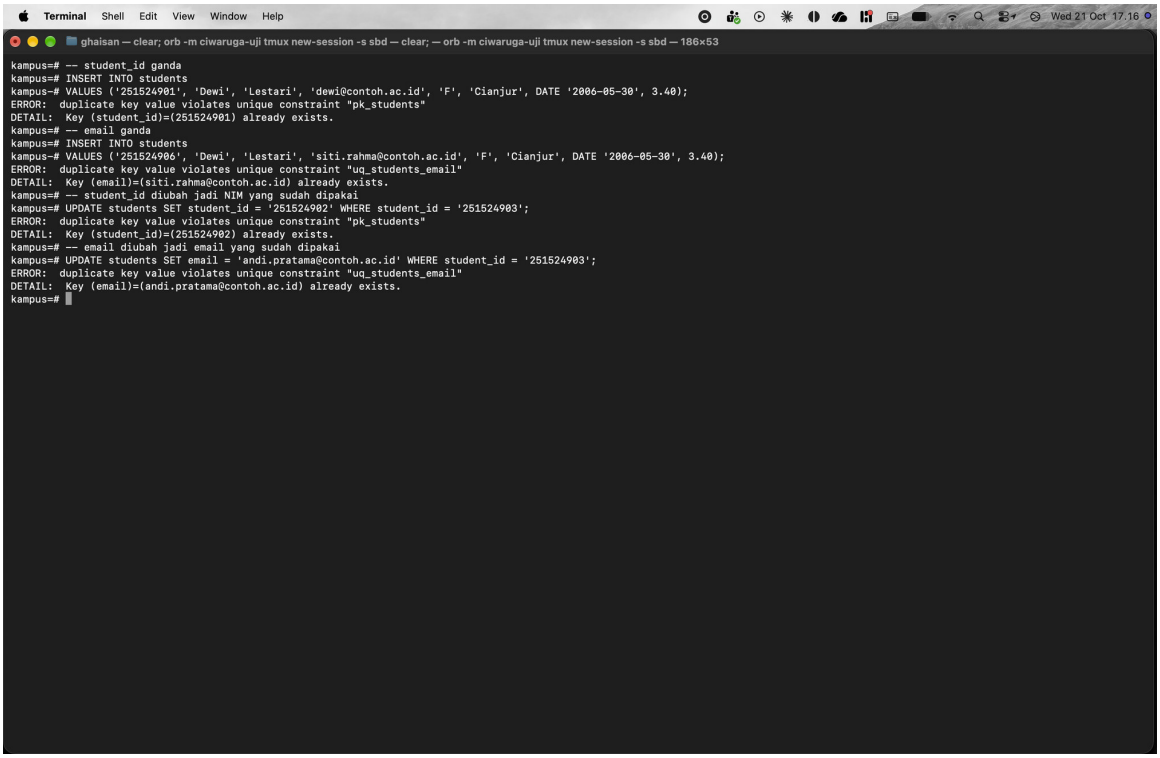
-- student_id diubah jadi NIM yang sudah dipakai
UPDATE students SET student_id = '251524902' WHERE student_id = '251524903';

-- email diubah jadi email yang sudah dipakai
UPDATE students SET email = 'andi.pratama@contoh.ac.id' WHERE student_id =
    '251524903';
```

Keterangan

INSERT dengan student_id 251524901 yang sudah dipakai Andi ditolak oleh pk_students dengan pesan duplicate key value dan keterangan Key (student_id)=(251524901) already exists.
INSERT dengan email siti.rahma@contoh.ac.id yang sudah dipakai Siti ditolak oleh uq_students_email.
UPDATE yang mengubah NIM atau email Rizky menjadi milik mahasiswa lain juga ditolak oleh constraint yang sama.
Hasil percobaan: berhasil. Key constraint PRIMARY KEY dan UNIQUE terbukti mencegah NIM dan email ganda (BR01, BR03).

Hasil Eksekusi (Screenshot)



Judul

Percobaan 6: entity integrity constraint (primary key tidak boleh NULL)

Syntax

```
-- student_id NULL
INSERT INTO students
VALUES (NULL, 'Dewi', 'Lestari', 'dewi@contoh.ac.id', 'F', 'Cianjur', DATE
    '2006-05-30', 3.40);

-- student_id tidak diisi
INSERT INTO students (first_name, last_name, email, gender, place_of_birth,
    date_of_birth)
VALUES ('Dewi', 'Lestari', 'dewi@contoh.ac.id', 'F', 'Cianjur', DATE
    '2006-05-30');

-- student_id dikosongkan lewat UPDATE
UPDATE students SET student_id = NULL WHERE student_id = '251524903';

-- data yang benar akhirnya bisa masuk
INSERT INTO students
VALUES ('251524906', 'Dewi', 'Lestari', 'dewi@contoh.ac.id', 'F', 'Cianjur', DATE
    '2006-05-30', 3.40);

SELECT * FROM students ORDER BY student_id;
```

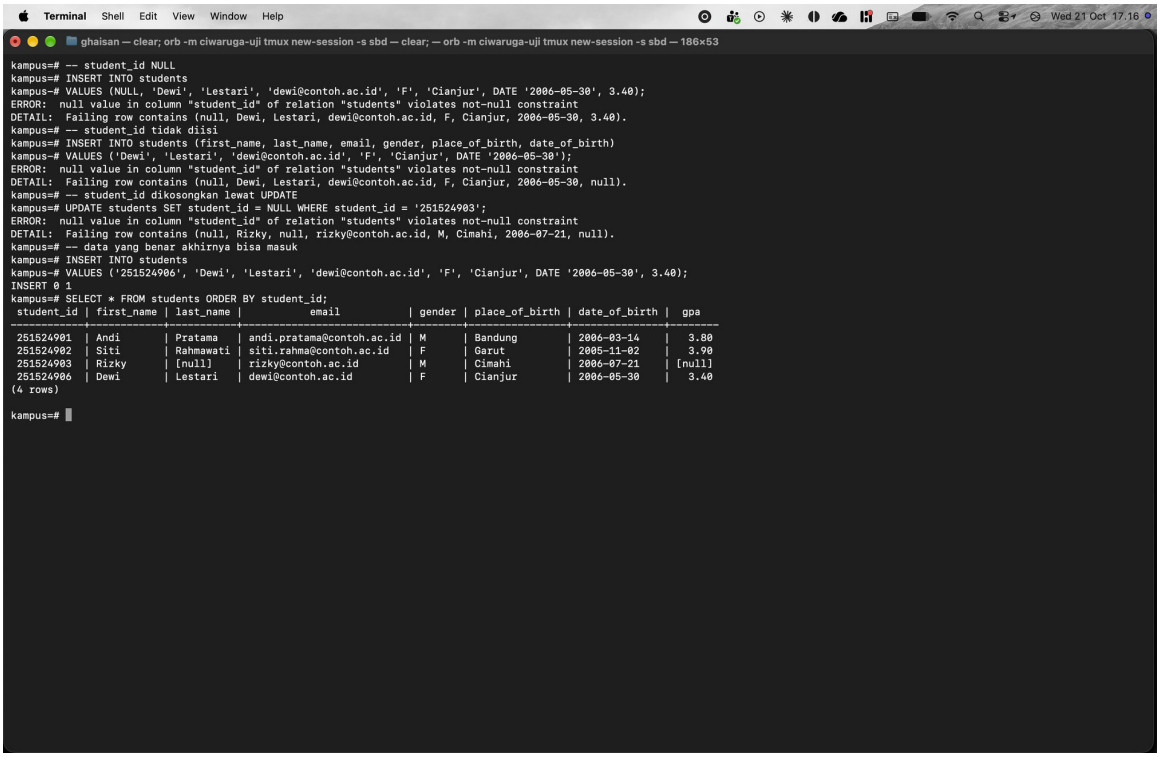
Keterangan

student_id NULL ditolak dengan pesan violates not-null constraint pada kolom student_id. Kolom ini tidak ditulis NOT NULL secara manual, tapi PRIMARY KEY di PostgreSQL otomatis berarti UNIQUE dan NOT NULL.

INSERT tanpa student_id dan UPDATE student_id menjadi NULL juga ditolak, jadi setiap baris pasti punya identitas.

Setelah datanya diperbaiki, Dewi dengan NIM 251524906 berhasil masuk (INSERT 0 1). Isi akhir tabel 4 baris: 3 dari Percobaan 1 dan 1 dari percobaan ini, dengan gpa Andi yang sudah berubah menjadi 3.80 dari Percobaan 4. Semua INSERT dan UPDATE yang ditolak tidak meninggalkan data apa pun.
Hasil percobaan: berhasil. Entity integrity constraint terbukti: primary key tidak boleh NULL.

Hasil Eksekusi (Screenshot)



6. Kesimpulan

Semua integrity constraint yang diminta, yaitu domain constraint (NULL dan CHECK), key constraint, dan entity integrity constraint, berhasil diterapkan pada tabel students. Setiap INSERT atau UPDATE yang melanggar business rule ditolak oleh PostgreSQL dengan pesan error yang menyebutkan nama constraint-nya, sedangkan data yang benar tetap bisa masuk. Semua percobaan berhasil, tidak ada percobaan yang gagal.